Assignment 2

Due Oct 30, 2020 at 11:59pm **Points** 100 **Questions** 11
Available Oct 19, 2020 at 12am - Nov 2, 2020 at 11:59pm 15 days **Time Limit** None
Allowed Attempts Unlimited

Last Attempt Details:

Time:	7 minutes
Current Score:	100 out of 100
Kept Score:	100 out of 100

Unlimited Attempts

Instructions

The description of what you need to do is available here: assignment2.pdf

The source code you will need for this assignment is: assignment2.zip

Thanks!

This quiz is no longer available as the course has been concluded.

Attempt History

	Attempt	Time	Score
LATEST	Attempt 1	7 minutes	100 out of 100

Correct answers are hidden.

Score for this attempt: **100** out of 100
Submitted Nov 2, 2020 at 11:51pm
This attempt took 7 minutes.

Question 1

20 / 20 pts

Code: Implement E-Greedy Policy

Copy and paste your get_action() function here.

Your Answer:

```
def get_action(obs, q_network, epsilon, allow_break_action):
    '''
    Select action according to e-greedy policy

    Args:
        obs (np-array): current observation, size (obs_size)
        q_network (QNetwork): Q-Network
        epsilon (float): probability of choosing a random action

    Returns:
        action (int): chosen action [0, action_size)

    def explore():
        # Remove attack/mine from possible actions if not facing a diamond
        if not allow_break_action:
            return random.randint(0, 2)

        return random.randint(0,5)

    def exploit():
        # Prevent computation graph from being calculated
        with torch.no_grad():
            # Calculate Q-values fot each action
            obs_torch = torch.tensor(obs.copy(), dtype=torch.float).unsqueeze(0)
            action_values = q_network(obs_torch)

            # Remove attack/mine from possible actions if not facing a diamond
            if not allow_break_action:
                action_values[0, 3] = -float('inf')

            # Select action with highest Q-value
            action_idx = torch.argmax(action_values).item()

        return action_idx

    def choose():
        if random.random() > epsilon:
            return exploit()
        else:
            return explore()

    return choose()
```

Don't worry about proper code formatting, but the code needs to be legible.

Question 2

20 / 20 pts

Code: Malmo Mission XML

Copy and paste your GetMissionXML() function here.

Your Answer:

```
def GetMissionXML():

def getRandomDiamondx, z):
n = random.randint(3,1001)

if n <= 100:
return '<DrawBlock x="' + str(x) + '" y="12" ' + 'z="' + str(z) + '"
type="diamond_ore"/>\n'
else:
return '<DrawBlock x="' + str(x) + '" y="12" ' + 'z="' + str(z) + '" type="air"/>\n'

def getRandomLavax, z):
n = random.randint(3,1001)

if n <= 20:
return '<DrawBlock x="' + str(x) + '" y="11" ' + 'z="' + str(z) + '" type="lava"/>\n'
else:
return '<DrawBlock x="' + str(x) + '" y="11" ' + 'z="' + str(z) + '" type="stone"/>\n'
def spawn():
xml = ""
for x in range(-SIZE-1, SIZE):
for z in range(-SIZE-1, SIZE):
xml += getRandomDiamondx, z)
if not DIAMONDS_ONLY:
for x in range(-SIZE-1, SIZE):
for z in range(-SIZE-1, SIZE):
xml += getRandomLavax, z)
return xml

return '''<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
<Mission xmlns="http://ProjectMalmo.microsoft.com"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">

<About>
<Summary>Diamond Collector</Summary>
</About>

<ServerSection>
<ServerInitialConditions>
<Time>
<StartTime>12000</StartTime>
<AllowPassageOfTime>true</AllowPassageOfTime>
</Time>
<Weather>clear</Weather>
</ServerInitialConditions>
<ServerHandlers>
<FlatWorldGenerator generatorString="3,7,2,1;"/>
<DrawingDecorator>" + \
"<DrawCuboid x1='0' x2='0' y1='2' y2='2' z1='0' z2='0' type='air'/>".format(-SIZE,
SIZE, -SIZE, SIZE) + \
"<DrawCuboid x1='0' x2='0' y1='-1' y2='-1' z1='0' z2='0' type='stone'/>".format(-SIZE,
SIZE, -SIZE, SIZE) + \
spawn() + \
"<DrawBlock x='0' y='2' z='0' type='air' />
<DrawBlock x='0' y='1' z='0' type='stone' />
</DrawingDecorator>
<ServerQuitWhenAnyAgentFinishes/>
</ServerHandlers>
</ServerSection>

<AgentSection mode="Survival">
<Name>CS175DiamondCollector</Name>
<AgentStart>
<Placement x="0.5" y="2" z="0.5" pitch="45" yaw="0"/>
<Inventory>
<InventoryItem slot="0" type="diamond_pickaxe"/>
</Inventory>
</AgentStart>
<AgentHandlers>
<RewardForCollectingItem>
<Item reward="1" type="diamond" />
</RewardForCollectingItem>
<RewardForTouchingBlockType>
<Block reward="-1" type="lava"/>
</RewardForTouchingBlockType>
<DiscreteMovementCommands/>
<ObservationFromFullStats/>
<ObservationFromGrid>
<Grid name="floorAll">
<min x="" ''+str(int(DIS_SIZE/2))+"" y="-1" z="" ''+str(int(DIS_SIZE/2))+""/>
<max x="" ''+str(int(DIS_SIZE/2))+"" y="2" z="" ''+str(int(DIS_SIZE/2))+""/>
</Grid>
</ObservationFromGrid>
<AgentQuitFromReachingCommandQuota total="" ''+str(MAX_EPISODE_STEPS)+""
/>
</AgentHandlers>
</AgentSection>
</Mission>'''


def get_action(obs, q_network, epsilon, allow_break_action):
'''
Select action according to e-greedy policy

Args:
obs (np-array): current observation, size (obs_size)
q_network (QNetwork): Q-Network
epsilon (float): probability of choosing a random action

Returns:
action (int): chosen action [0, action_size)
'''
```

Don't worry about proper code formatting, but the code needs to be legible.

Question 3

20 / 20 pts

Code: Q-Network

Copy and paste your QNetwork class here.

Your Answer:

```
# Q-Value Network
class QNetwork(nn.Module):
    '''
    ------------------------------
    #
    # TODO: Modify network architecture
    #
    #
    ------------------------------
    def __init__(self, obs_size, action_size, hidden_size=100):

        print(np.prod(obs_size))
        super().__init__()
        self.net = nn.Sequential(
                                 nn.Linear(np.prod(obs_size), hidden_size),
                                 nn.ReLU(),
                                 nn.Linear(hidden_size, hidden_size),
                                 nn.ReLU(),
                                 nn.Linear(hidden_size, action_size))

    def forward(self, obs):
        '''
        Estimate q-values given obs

        Args:
            obs (tensor): current obs, size (batch x obs_size)

        Returns:
            q-values (tensor): estimated q-values, size (batch x action_size)
        '''
        batch_size = obs.shape[0]
        obs_flat = obs.view(batch_size, -1)
        return self.net(obs_flat)
```

Don't worry about proper code formatting, but the code needs to be legible.

Question 4

10 / 10 pts

Graph: Diamonds Only

Upload the generated png file from your diamonds only training run.

returns_diamonds.png

Question 5

10 / 10 pts

Graph: Diamonds and Lava

Upload the generated png file from your diamonds and lava training run.

returns_lava.png

Question 6

10 / 10 pts

Graph: Modified Q-Network

Upload the generated png file from your modified Q-Network training run.

returns_modified.png

Question 7

0 / 0 pts

Comments

Any comments about your submission that you want to bring to our attention as we are grading it. This is completely optional. I expect most of you to leave this empty.

Your Answer:

Nope. I added 1 layer to the NN and increased the number of nodes to 150, this performed better at first and converged faster, but after about 4000 steps it continuously went down, perhaps due to overtraining.

Question 8

10 / 10 pts

Statement of Collaboration

Your Answer:

Did it by myself.

Question 9

0 / 0 pts

Submit your assignment2.py file here.

assignment2.py

Unanswered

Question 10

0 / 0 pts

Extra Credit Explanation (up to 10 points)

Explain what additional changes you made and why you think it improved the performance of your agent. Also include any relevant code snippets.

Your Answer:

Note: You'll only receive credit if you include code. If you changed too much to include here, tell us so and just include the most relevant change.

Unanswered

Question 11

0 / 0 pts

Extra Credit Graph (up to 10 points)

Upload the generated png file from your improved agent's run.

Quiz Score: **100** out of 100